

RESEARCH ARTICLE

High-Performance Unified Hardware Architecture for ML-DSA and ML-KEM PQC Standards

QUANG DANG TRUONG¹, (Graduate Student Member, IEEE),

YUNSEONG JANG¹, (Graduate Student Member, IEEE),

AND HANHO LEE¹, (Senior Member, IEEE)

Department of Electrical and Computer Engineering, Inha University, Incheon 22212, South Korea

Corresponding author: Hanho Lee (hlee@inha.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP)-Information Technology Research Center (ITRC) grant funded by Korean Government Ministry of Science and ICT (MSIT) under Grant RS-2021-II212052, in part by the National Research and Development Program through the National Research Foundation of Korea (NRF) funded by MSIT under Grant RS-2020-NR047143, and in part by the Commercialization Promotion Agency for Research and Development Outcomes (COMPA) funded by MSIT under Grant RS-2025-02219156.

ABSTRACT In response to the advent of quantum computers, which pose significant threats to traditional cryptographic systems, NIST announced the quantum-resistant Module Lattice-based Key Encapsulation Mechanism (ML-KEM) and Digital Signature (ML-DSA) standards in August 2024. While prior studies have proposed hardware designs to accelerate either ML-KEM or ML-DSA individually, a unified architecture supporting all requirements of both schemes remains lacking. Leveraging the shared lattice-based cryptographic structures of ML-KEM and ML-DSA, this paper proposes a novel, high-performance architecture that supports all phases of both standards. The design allows flexible switching between operation modes and security levels at runtime, providing versatility and user-friendliness. Based on a long-term analysis of both cryptosystem theories, we propose optimizations in unified components, with specific configurations that execute both schemes efficiently in hardware while reducing redundant resource utilization from overlapping tasks. The timing diagram is carefully arranged to maximize concurrency, allowing the architecture to deploy multiple tasks in parallel, thereby addressing performance bottlenecks commonly seen in asymmetric cryptosystems. Moreover, we integrate several efficient countermeasure techniques to mitigate side-channel attack vulnerabilities in the hardware implementation. The proposed architecture is validated through implementation on Zynq UltraScale+ FPGA platform, demonstrating optimal execution times for each main algorithm in both post-quantum cryptography standards. Compared to state-of-the-art designs, our implementation exhibits superior performance and efficiency, as evaluated by the area-time product metric, as well as solves problems arising from outdated standards in previous studies.

INDEX TERMS Post-quantum cryptography (PQC), lattice-based cryptography, digital signature standard, key encapsulation mechanism (KEM), polynomial arithmetic architecture, secure hash algorithm.

I. INTRODUCTION

In recent years, quantum computing has advanced significantly and is expected to become practically available in the near future. With the potential for substantial performance gains, quantum computing can efficiently solve

complex mathematical problems that underpin widely used cryptographic algorithms such as RSA and elliptic curve cryptography. To address these emerging threats, the National Institute of Standards and Technology (NIST) published the Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM) and Module-Lattice-Based Digital Signature Algorithm (ML-DSA) standards in August 2024, derived from the CRYSTALS-Kyber and Dilithium algorithms,

The associate editor coordinating the review of this manuscript and approving it for publication was Shuangqing Wei¹.

respectively, following an extensive evaluation and standardization process [11]. These standards were formalized as Federal Information Processing Standards Publications (FIPS) 203 and FIPS 204.

As lattice-based cryptosystems, ML-KEM and ML-DSA rely on the hardness of the Module Learning With Errors (MLWE) problem, which is believed to be secure even against adversaries equipped with quantum computers [2], [3]. The MLWE problem, similar to the LWE problem, involves computationally intensive sampling and polynomial arithmetic operations. Additionally, to ensure security, the Secure Hash Algorithm 3 (SHA-3) [4] is employed in both schemes as the cryptographic hash function. With the potential to become new standards, replacing current cryptosystems in cryptographic applications, there is a growing interest in developing efficient implementations for these cryptosystems in both software and hardware as accelerators. Given that ML-KEM and ML-DSA encompass three main algorithms and offer three parameter sets corresponding to different security categories, implementations of these schemes require flexibility and versatility to switch between operation modes, enhancing user-friendliness. Therefore, hardware implementations, particularly on FPGA platforms, present a promising option for achieving low-cost and high-performance post-quantum cryptographic solutions, including ML-KEM and ML-DSA.

During the NIST post-quantum cryptography (PQC) standardization process, numerous hardware implementations for CRYSTALS-Kyber and Dilithium have been proposed. To the best of our knowledge, the most notable ones are summarized here [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25]. In early 2021, Ricci et al. introduced both high-level synthesis and register-transfer level (RTL) implementations for the each main Dilithium algorithms on FPGAs [5]. Subsequently, reconfigurable Dilithium architectures tailored for each security category were proposed in pure hardware [6] and in hardware-software co-design frameworks [11]. Later, C. Zhao and H. Zhao presented incomplete Dilithium hardware architectures [7], [8]. Although these designs support the core Dilithium algorithms, they lack essential packing and unpacking modules, as well as keys and signature transition processes. As a result, all operations remain confined to on-board execution, limiting the practicality of communication in public-key cryptosystems. Comprehensive hardware architectures addressing these limitations—supporting all main algorithms, security levels and including AXI stream interfaces and data transition processes—were reported in [9], [10], and [13]. More recently, Truong et al. proposed a low-latency architecture supporting the draft version of ML-DSA [14].

Regarding Kyber, over 30 papers have introduced various hardware architectures for its implementation. To the best of our knowledge, Bisheh-Niasar et al. [17] presented the first instruction-set of accelerated implementations for each Kyber

security category. Ni et al. [18] proposed high-performance Kyber accelerators utilizing fully-pipelined Number Theoretic Transform (NTT) architectures; however, these designs independently support each security category and lack the configurability and versatility of designs that support all security categories within a single framework. More recent works have focused on highly efficient [20] and lightweight [21], [22] configurable hardware architectures aimed at minimizing hardware area usage for Kyber implementations. Notably, in 2023, Kim et al. [23] proposed a configurable ML-KEM accelerator architecture aligned with the draft version of ML-KEM. Most recently, Nguyen et al. [24] and Jung et al. [25] have proposed configurable ML-KEM architectures supporting all algorithms and security levels.

Given their shared lattice-based structure, Dilithium and Kyber algorithms possess commonalities that can be leveraged to develop a unified hardware design. Such integration can reduce the cost associated with implementing multiple independent hardware solutions and enhance flexibility for users. Despite differences in modulus q within $\mathbb{Z}_q$, the arithmetic operations and pseudorandom sampling processes in both schemes can be efficiently integrated into a single hardware module, as demonstrated in recent studies [26], [27]. For unified hardware architectures supporting both digital signatures and key encapsulation mechanisms, Aikata et al. proposed cryptoprocessors integrating Dilithium and Saber [28], as well as Dilithium and Kyber [29]. However, following the announcement of the new standards for ML-DSA and ML-KEM by NIST, these cryptoprocessors have become outdated due to potential risks arising from incomplete standardization, necessitating updates to align with the new standards.

To efficiently implement cryptosystems on hardware, a well-designed timing schedule is crucial. It ensures that submodules operate without conflicts and achieve optimal execution time. Given their lattice-based structure, both ML-DSA and ML-KEM involve computationally intensive processes related to polynomial arithmetic operations, hashing and sampling. Therefore, efficient implementations must coordinate hash and arithmetic modules effectively, minimizing idle periods in the timing schedule to maintain continuous operation as much as possible. In this context, previous studies [28], [29] have focused solely on the instruction-set design of submodules, without considering proposed timing schedules or how to combine submodules efficiently within them. Moreover, ML-DSA.Sign follows Fiat-Shamir With Aborts paradigm (F-SWA) [30], which requires a rejection sampling loop when deriving a “challenge” c [3]. Implementing this procedure in hardware is challenging and demands significant effort to limit the execution time of signature generation using the F-SWA paradigm. However, solutions to this issue are often overlooked in previous studies, including [28], [29]. Additionally, regarding unified hardware design for both standards, there is a lack of high-performance implementation approach, which

is necessary to optimize hardware efficiency and address the execution time bottleneck of post-quantum cryptographic systems in the internet-era requirements.

In addition to the need for optimized hardware designs that achieve high performance and competitive resource utilization, cryptographic implementations must also ensure security in practical applications. Previous studies have identified vulnerabilities in the Kyber and Dilithium algorithms to side-channel attacks (SCAs) [31], [32], [33]. Implementations that execute the Kyber and Dilithium algorithms step by step, following the order of these algorithms, are particularly vulnerable to electromagnetic (EM) and power analysis attacks. Furthermore, side-channel leakage during Barrett reduction, as observed in [34], must also be considered, given its common usage in previous studies. However, these issues have often been overlooked in prior hardware implementations for Kyber and Dilithium. Although ML-DSA and ML-KEM have been enhanced from their derived versions to improve security, it remains crucial to consider these physical attacks when designing secure implementations. Addressing these concerns is essential to mitigate risks during key exchange and digital signature verification in KEM and DSA applications.

In this paper, we propose a high-performance hardware implementation that integrates both ML-DSA and ML-KEM on the Xilinx Zynq UltraScale+ FPGA platform. Our design supports all primary algorithms of both cryptographic standards and allows dynamic switching between operation modes and NIST-defined security categories at runtime. To the best of our knowledge, this is the first unified hardware implementation fully aligned with the final versions of the two new NIST standards for key encapsulation and digital signature, validated against the official test vectors provided by the NIST Cryptographic Algorithm Validation Program [35]. By leveraging flexible, reusable submodules and a carefully tailored timing schedule specific to ML-DSA and ML-KEM, our design achieves optimal performance across all algorithms and security categories. Furthermore, we address concerns regarding vulnerabilities to SCAs identified in previous studies by scheduling our submodules to operate concurrently in parallel as much as possible, thereby optimizing execution time and limiting information leakage that could be exploited by SCAs. Furthermore, we propose unified optimizations for common submodules shared between both schemes, reducing on-chip resource utilization without sacrificing speed. The complete architecture requires 67.3k LUTs, 23.7k FFs, 16 DSPs, and 27.5 BRAMs—comparable to prior efficient Dilithium-only or unified implementations—while achieving the best performance reported to date for both Kyber and Dilithium hardware accelerators. The key contributions of this work can be summarized as follows:

- 1) We propose the first high-performance unified hardware architecture that supports all main algorithms of ML-DSA and ML-KEM. This design is configurable

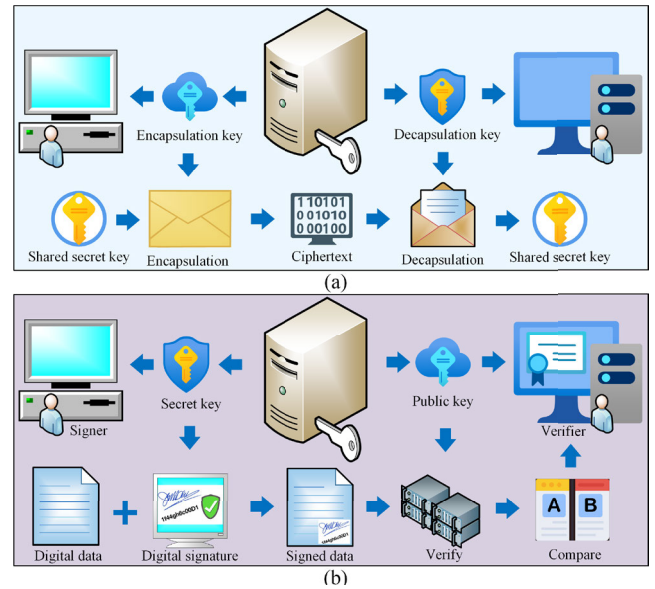


FIGURE 1. Overview of operations for (a) Key-Encapsulation mechanism and (b) Digital signature.

to flexibly switch between operation modes and NIST security categories. To achieve high performance, we propose an appropriate timing schedule that optimizes the cooperation between submodules within the combined hardware architecture. Furthermore, proposed design includes packing/unpacking modules that enable efficient data exchange with other platforms, facilitating accelerated computational tasks in both cryptosystems.

- 2) An approach utilizing multiple unified submodules, specifically the Hash and unified polynomial arithmetic module (UniPAM), is introduced. This strategy improves the execution time of each main algorithm in both cryptosystems, optimizes the timing schedule, and enhances security by mitigating information leakage from SCAs during specific computation periods, which can occur when using a single submodule for each task.
- 3) We present novel proposals to integrate two cryptosystems into a single design by leveraging their commonalities to create configurable versatile submodules, particularly the UniPAM, memory, en/decoder and Hash modules. These submodules are efficiently configurable to operate in multiple modes, reducing the need for independent submodules, which could otherwise increase unnecessary hardware resource requirement and complicate control management without enhancing overall architecture performance.
- 4) Finally, we evaluate the proposed unified architecture on the Xilinx Zynq UltraScale+ ZCU102 FPGA platform and compare it with state-of-the-art works. Our implementation results demonstrate that the proposed design achieves superior performance. The efficiency, evaluated by Area-Time Product metric, shows significant improvements over previous unified studies.

The remainder of this paper is structured as follows. An overview of ML-KEM and ML-DSA, along with their auxiliary functions, is provided in Section II. Section III presents our design decisions, including schedule diagrams for each algorithm, and details our hardware architecture as well as the optimized modules. Section IV discusses the implementation results and compares them with state-of-the-art works. Finally, Section V summarizes and concludes the paper.

II. PRELIMINARIES

A. MODULE-LATTICE-BASED KEY ENCAPSULATION MECHANISM

ML-KEM, derived from CRYSTALS-Kyber, is a lattice-based KEM designed to enable two parties to establish shared secret key over public channels. Its security is grounded in the computational hardness of MLWE problem, which is believed to be resistant even against quantum adversaries [2]. The ML-KEM scheme comprises 3 main algorithms: key generation (KeyGen), encapsulation (Encap), and decapsulation (Decap). To balance security and efficiency, the standard specifies three parameter sets corresponding to different security categories, detailed in Section 8 of FIPS 203. In summary, ML-KEM construction is presented through its main algorithms as detailed in [2] and illustrated in general in Figure 1:

1) KEYGEN

This algorithm generates a key pair consisting of an encapsulation key (ek) and a decapsulation key (dk) from a random seed. The decapsulation key includes the secret vector $\mathbf{s}$, while the encapsulation key comprises a collection of “noisy” linear equations ($\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}$) in the secret variables $\mathbf{s}$. The process begins by pseudorandomly generating the public matrix $\hat{\mathbf{A}}$, secret vector $\mathbf{s}$ and noise $\mathbf{e}$. After that, the computation of the final part $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ is computed to form part of the encapsulation key. Both keys are then appropriately encoded into byte arrays and output as ek and dk.

2) ENCAPULATION

Given an encapsulation key and a 32-byte plaintext message m , this algorithm produces a ciphertext c and shared key K . It begins by extracting the vector $\mathbf{t}$ and the seed from the ek. A copy of the shared secret key K is derived from m and ek via hashing. The algorithm proceeds by generating a vector $\mathbf{y}$ and noise term $\mathbf{e}_1, \mathbf{e}_2$. It then computes the “new noisy equation”, which is $(\hat{\mathbf{A}}^\top \mathbf{y} + \mathbf{e}_1, \mathbf{t}^\top \mathbf{y} + \mathbf{e}_2)$. An appropriate encoding μ of m is added to the latter term. Finally, the resulting pair $(\mathbf{u}, \mathbf{v})$ is compressed, serialized into a byte array, and output as the ciphertext.

3) DECAP

This algorithm takes the dk and c as input to derive the shared secret key. It decrypts the ciphertext to recover a plaintext m' . The algorithm then re-encrypts m' to compute a

TABLE 1. Summary of the NTT architectures for ML-KEM and ML-DSA.

Ref	Structure	Resource Utilization				Freq ¹ (MHz)	Latency ² (CCs)
		LUT	FF	DSP	RAM		
[37]*	R2MDC	4,834	4,683	0	1	250	128/128
[38]*	R2MDF	3,918	4,292	26	1	265	64/64
[37]**	R2MDC	7,451	5,275	0	0	180	128/128
[37]**	R4MDC	13,804	11,019	0	0	163	64/64
[40]*	Radix-2	2,543	792	4	9	182	232/233
[41]*	Radix-2	2,081	2237	0	0	304.7	504/504
[20]*	Radix-2/4	1,740	643	4	0	200	224/224
[42]**	Radix-4	2,637	1,071	8	1	385	268/268
[44]*	Radix-8/16	9,985	4,580	29	12	132	58/58
[44]**	Radix-16	12,720	7,834	50	14.5	183	58/58

*/**: Supporting for NTT of Kyber/ Dilithium.

¹: Except [42] installed on Zynq US+, others on Artix-7 platform.

²: Computation of NTT/INTT for one polynomial.

candidate shared secret key K' in the same manner as during encapsulation. It checks whether the resulting ciphertext c' matches the provided c . If they do not match, the algorithm performs an “implicit rejection” by replacing K' with a hash of c and a random value z stored in the secret key. In either case, the algorithm outputs the resulting shared secret key K' .

B. MODULE-LATTICE-BASED DIGITAL SIGNATURE ALGORITHM

ML-DSA is a digital signature scheme derived from CRYSTALS-Dilithium, a lattice-based algorithm designed to be secure even against adversaries possessing large-scale fault-tolerant quantum computers. Its security is based on the hardness of MLWE and Short Integer Solution problems [3]. The scheme comprises 3 primary algorithms: key generation (KeyGen), signature generation (Sign), and signature verification (Verify). To balance security and performance, ML-DSA is standardized with 3 parameter sets, each aligned with different NIST security categories, as specified in FIPS 204. In summary, ML-DSA construction is defined through its three main algorithms and is illustrated in general in Fig. 1:

1) KEYGEN

The key generation process produces a key pair consisting of a public verification key (pk) and a private signing key (sk). It utilizes an input seed ξ to expand into a public random seed ρ and a private random seed ρ' , which are used to pseudorandomly sample matrix $\hat{\mathbf{A}}$ and vectors $\mathbf{s}_1, \mathbf{s}_2$. It then computes $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ to generate the final key pair.

2) SIGN

This process takes a message M , random seed rnd , context string ctx and sk to generate a signature attached to the message before sending it to verifiers. Sign phase begins by sampling a masking vector $\mathbf{y}$, public matrix $\hat{\mathbf{A}}$ and then computing $\mathbf{w} = \mathbf{A}\mathbf{y}$. A challenge c is generated by hashing the message with the high-order bits of $\mathbf{w}$. The signature component $\mathbf{z}$ is computed as $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, along with hints

$\mathbf{h}$ for the verifier. The potential signature σ consisting of $(\tilde{c}, \mathbf{z}, \mathbf{h})$, is subjected to a rejection sampling loop to ensure it meets the required conditions. This loop repeats until a valid signature is produced, which is then encoded as a byte string and output.

3) VERIFY

This process takes a public key, a message, and a signature as input. It reconstructs the signer's commitment $\mathbf{w}_1$ from the pk and σ , referred to as $\mathbf{w}'_1$. Together with the message representative μ , it is used as a seed for a hash function to generate a value $\tilde{c}'$, which is compared to the signer's commitment hash $\tilde{c}$ provided in the signature to confirm the authenticity and integrity of the original message.

C. POLYNOMIAL ARITHMETIC OVER FINITE FIELD

These two standards make extensive use of arrays of integers modulo q (i.e., elements of $\mathbb{Z}_q^n$). Each standard represents an element of the ring R_q as a polynomial containing n coefficients. The values n and q are two global integer constants, with $n = 256$ in both standard, while $q = 3329$ in ML-KEM and 8380417 in ML-DSA. Algebraic operations such as addition and scalar multiplication are performed coordinate-wise. Therefore, both standards utilize the NTT, a variant of the Fast Fourier Transform that operates over a finite field, to reduce the complexity of polynomial multiplication from $\mathcal{O}(n^2)$ using the school-book method to $\mathcal{O}(n \log n)$. Specifically, the mathematical structure of the NTT in ML-KEM involves 128 primitive 256-th roots of unity in $\mathbb{Z}_q$. The coordinate-wise multiplication (CWM) operation in ML-KEM is defined in the specification [2] as follows: $\hat{h}_{2i} + \hat{h}_{2i+1}X = (\hat{f}_{2i} + \hat{f}_{2i+1}X) \cdot (\hat{g}_{2i} + \hat{g}_{2i+1}X) \bmod (X^2 - \zeta^{2br(i+1)})$. This computation can be optimized using the Karatsuba algorithm, reducing number of required multiplications from five to four:

$$\begin{aligned} \hat{h}_{2i} &= \hat{f}_{2i}\hat{g}_{2i} + \hat{f}_{2i+1}\hat{g}_{2i+1} \cdot \zeta^{2br(i+1)} \\ \hat{h}_{2i+1} &= (\hat{f}_{2i} + \hat{f}_{2i+1})(\hat{g}_{2i} + \hat{g}_{2i+1}) - \hat{f}_{2i}\hat{g}_{2i} - \hat{f}_{2i+1}\hat{g}_{2i+1} \end{aligned} \quad (1)$$

With their lattice-based structure, both standards involve extensive arithmetic operations. Consequently, the UnipAM is a critical submodule that determines the efficiency of the entire hardware implementation. Two typical variants of the NTT architecture are commonly used: (i) fully-pipelined and (ii) in-place architectures. In the fully pipelined approach, all stages are processed sequentially, outputting data every clock cycle (CC) to achieve high-speed performance. In contrast, the in-place approach utilizes one or multiple processing elements (PEs) to perform computations iteratively, stage by stage. Therefore, the fully pipelined architecture is well-suited for high-performance implementations prioritizing execution time, while the in-place approach is more appropriate for resource-efficient hardware designs.

In case of fully-pipelined architectures [36], two typical designs are often employed: the multipath delay commutator

(MDC) architecture, applied to Kyber and Dilithium in [37], and the multipath delay feedback (MDF) architecture, used for Kyber in [38]. While these architectures offer high-speed performance, they require at least eight and seven PEs to handle the NTT operations of Dilithium and Kyber, respectively. Additionally, they necessitate independent modules for polynomial multiplication tasks, which can lead to inefficiencies in terms of hardware resource consumption. The in-place NTT architecture is commonly employed in efficient hardware implementations that prioritize minimal hardware resource consumption. For 256-point polynomials, as utilized in CRYSTALS-Kyber and Dilithium, radix-2 (R2) and radix-4 (R4) NTT methods are prevalent. These methods were introduced in [39] and have been applied in Kyber [40], [41] and Dilithium [42]. This approach directly connects PEs with memory used to store polynomial samples, making it ideal for integration of NTT with other polynomial arithmetic operations (PAOs) into compact, resource-efficient implementations. However, it requires a conflict-free memory access pattern, as mentioned in [43], which can increase the complexity of control logic and memory usage. To boost performance, high-radix methods are used at the cost of higher area utilization, e.g., radix-16 NTT for Dilithium and mixed radix-8/16 for Kyber [44]. Beyond these two typical architectures, Truong et al. [45] recently introduced a hybrid NTT architecture that combines the aforementioned methods. A summary of proposed NTT architectures, along with their resource utilization and latency, is presented in Table 1.

D. CRYPTOGRAPHIC FUNCTIONS

These two standards require the use of several cryptographic functions, which are instances of SHA-3 as specified in FIPS 202 [4]. These functions serve different purposes. The eXtendable-Output Function (XOF) is used in both standards to expand public samples, specifically utilizing the SHAKE128 function. SHAKE256 is employed to expand secrets, noise, and masking vectors in both standards. In ML-DSA, the hash function, typically denoted as H , also uses SHAKE256. In contrast, ML-KEM defines three hash functions: H as SHA3-256, J as SHAKE256 and G as SHA3-512. Therefore, total of four SHA-3 instances are used across both standards. All these functions share the same underlying sponge construction, in which input data is absorbed into “sponge” and then squeezed out to produce output. The core of the sponge construction is the permutation function known as *Keccak-p*, which is common to all SHA-3 instances. As a result, in hardware architecture, hash functions are typically designed to be configurable across multiple modes. This configurability allows reuse of the shared sponge construction and significantly reduces on-chip resource utilization for related cryptographic operations.

III. HIGH-LEVEL CONFIGURABLE UNIFIED ML-DSA AND ML-KEM ARCHITECTURE

As the design aims for high-performance implementation for the ML-DSA and ML-KEM standards on FPGA SoC

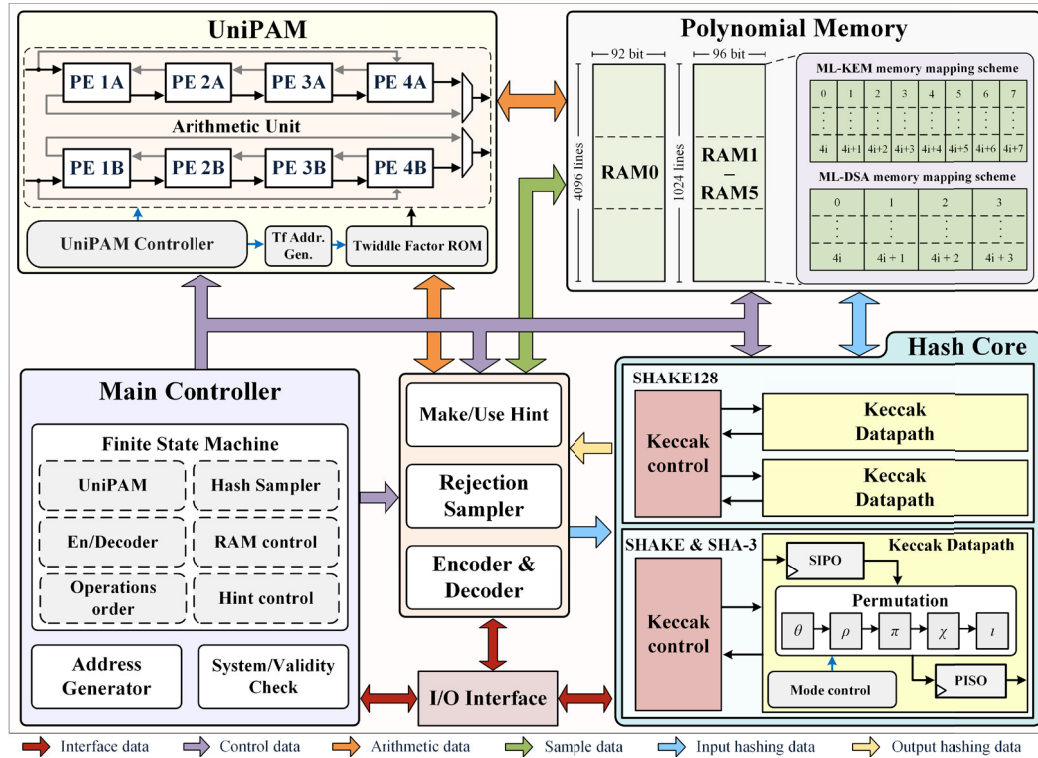


FIGURE 2. High-level configurable unified ML-DSA and ML-KEM architecture.

platforms, the proposed unified architecture supports all primary algorithms across the 3 security categories defined in the specifications. This design encompasses all processes, including the packing and transmission of ciphertexts, signatures, and key pairs, thereby enhancing flexibility and user-friendliness by facilitating seamless integration with other hardware and software platforms. This section introduces the overall high-level architecture and details the operation and purpose of its submodules. To complement the efficient hardware implementation, timing schedules depicted in Figs. 3–6 are proposed to optimize the operation of the entire architecture. These schedules aim to achieve high performance by strategically arranging tasks to leverage the most critical and time-consuming submodules, specifically the UniPAM and hash modules configured for both schemes.

A. OVERALL HARDWARE ARCHITECTURE AND OPERATION SCHEDULING

1) OVERALL HARDWARE ARCHITECTURE

The high-level architecture of our unified hardware design is depicted in Fig. 2. This configurable architecture supports the three primary algorithms in both NIST standards across all specified security levels. Fig. 2 illustrates the roles, interconnections, and interactions of the components constituting the complete architecture. To ensure compatibility with external systems, the design incorporates packing and unpacking processes, handling data in 8-byte (64-bit) segments. This facilitates seamless integration with various hardware and software platforms. Internally, the architecture operates with

a 96-bit bus width, enabling simultaneous processing of four coefficients for ML-DSA or eight coefficients for ML-KEM. Except for the control unit, UniPAM, and hash module—which are utilized across all algorithms—other submodules are connected as needed, depending on the specific requirements of each cryptographic operation. A brief description of these modules and their functionalities is provided below.

a: CONTROL UNIT

The control unit plays a vital role in overseeing other modules, ensuring the connection between submodules. Functioning as a finite-state machine (FSM), it manages these operations based on required functionalities and sequences the entire hardware module to execute all the primary algorithms of both standards in their respective orders. To configure and flexibly switch operation modes, two control signals: ‘mode’ and ‘sec_lvl’, are used to determine which algorithm and security category the architecture operates in. Depending on these signals, parameters are adjusted in accordance with the specifications outlined in [2] and [3]. Randomness seeds, messages, and other inputs are received from users during operations without being stored post-processing, thereby enhancing security and mitigating risks of SCAs, fault attacks or information leakage due to memory vulnerabilities.

b: POLYNOMIAL MEMORY

This module comprises six groups of three dual-port 36Kb BRAMs used to store intermediate values during computation. In ML-DSA and ML-KEM schemes, modulus q

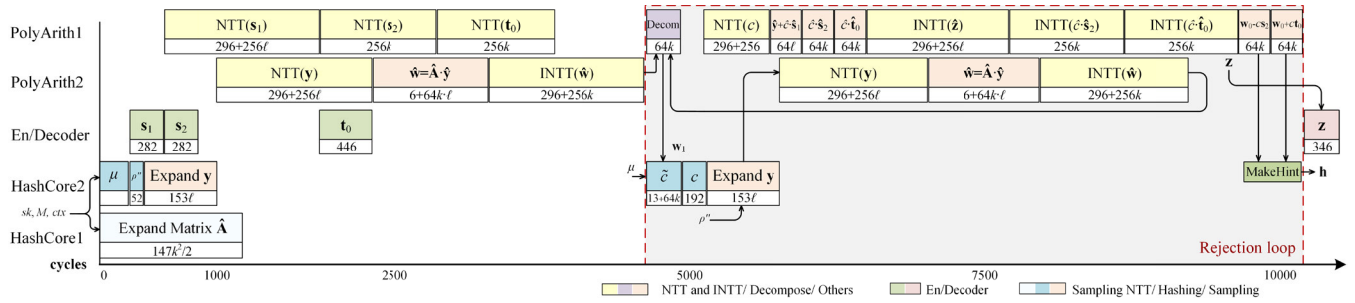


FIGURE 3. Detailed timing schedule of the optimized execution data flow for ML-DSA.Sign at security level 2.

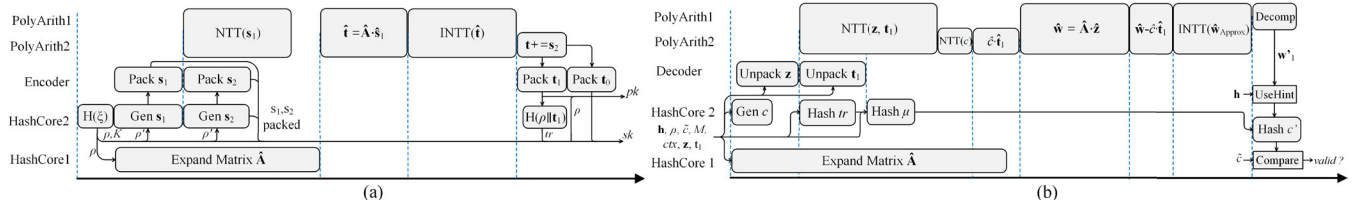


FIGURE 4. Timing schedule for ML-DSA-87: (a) Key generation (ML-DSA.Keygen) and (b) Verification (ML-DSA.Verify).

allows coefficients to be represented within 23 and 12 bits, respectively. Leveraging Xilinx FPGA's dual-port 36Kb BRAMs [46], each group is configured as 1024×96 memory. This setup enables storage of four ML-DSA or eight ML-KEM coefficients (requiring 96 bits) in a single address, achieving a 25% reduction in BRAM utilization compared to the straightforward approach of storing each coefficient in separate banks, as seen in prior Dilithium [11] and ML-KEM [25] architectures. The memory construction is demonstrated as memory mapping scheme outlined in Fig. 2. Due to the substantial size of public matrix $\hat{A}$ in ML-DSA, RAM0 is exclusively designated for storing this matrix. It is configured as a 92×4096 memory, consuming approximately 10.5 BRAMs. Additionally, five 96×1024 -bit BRAMs are employed to store intermediate values during the computation processes in both schemes.

c: UniPAM

UniPAM encompasses dual polynomial arithmetic units (PolyAriths) responsible for all arithmetic operations, including NTT, INTT, CWM, additions, subtractions, decompose and compressions across both standards.

d: HASH MODULE

As illustrated in Fig. 2, the Hash module comprises two Hash Cores: one specifically designed to handle the SHAKE128 function, and the other configurable to support three instances of the SHA-3 standard. These modules generate hash values from messages, random seeds, and pseudorandom data, which are subsequently used for polynomial sampling and hashing in both schemes.

e: SAMPLING UNIT (REJECTION SAMPLER)

Directly connected to the hash module, this unit includes submodules configured with rejection criteria derived from

pseudorandom procedures in both standards. It generates algebraic objects pseudorandomly, including public matrix $\hat{A}$, secret, noise, mask vectors and challenge c used during algorithm execution.

f: HINT MODULE

To support signature and key size compression, this module comprises two submodules: "MakeHint" and "UseHint", corresponding to their respective functions. The "Hint", denoted as h , is an integral component of signature, enabling verifier to reconstruct sufficient information from the dropped public-key bits to verify the signature.

g: ENCODER/DECODER

Responsible for packing and unpacking, it handles output data such as key pairs, signatures, and ciphertexts, packaging them into 8-byte segments for transmission, aligning with typical data interfaces between hardware and software platforms. The encoder packs polynomial vectors into byte strings, with different vectors and their corresponding bit widths consolidated into fixed 8-byte strings. Conversely, decoder retrieves the original vectors from these byte strings before used in subsequent computational phases.

Unlike software implementations, our hardware design allows for concurrent handling of multiple procedures, provided they do not conflict, depending on the algorithm being executed. For instance, in KeyGen of both schemes, upon receiving public and private seeds, the two Hash Cores can simultaneously generate samples. Such parallelism is meticulously arranged, leveraging all configurable submodules within our architecture to maximize execution speed for each algorithm, as demonstrated in the timing schedules presented in Figs. 3–6. Based on these schedules and the characteristics of the ML-KEM and ML-DSA algorithms, polynomial generation and computation phases are identified

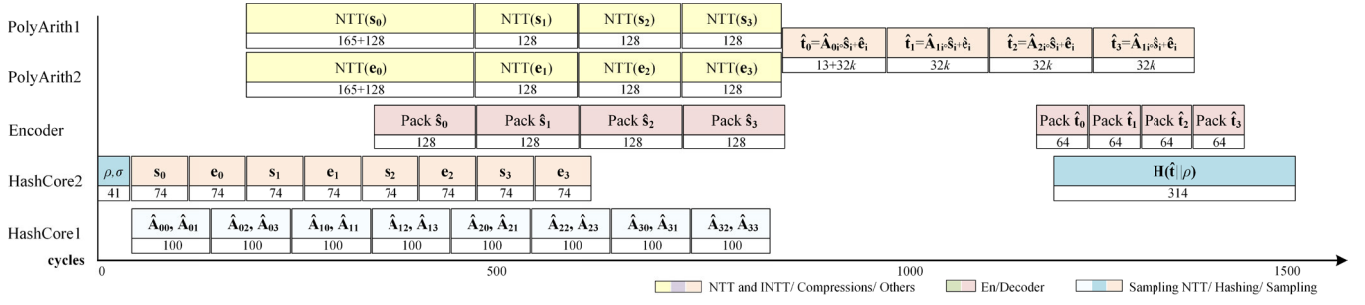


FIGURE 5. Detailed timing schedule of the optimized execution data flow for ML-KEM.Keygen at security level 5.

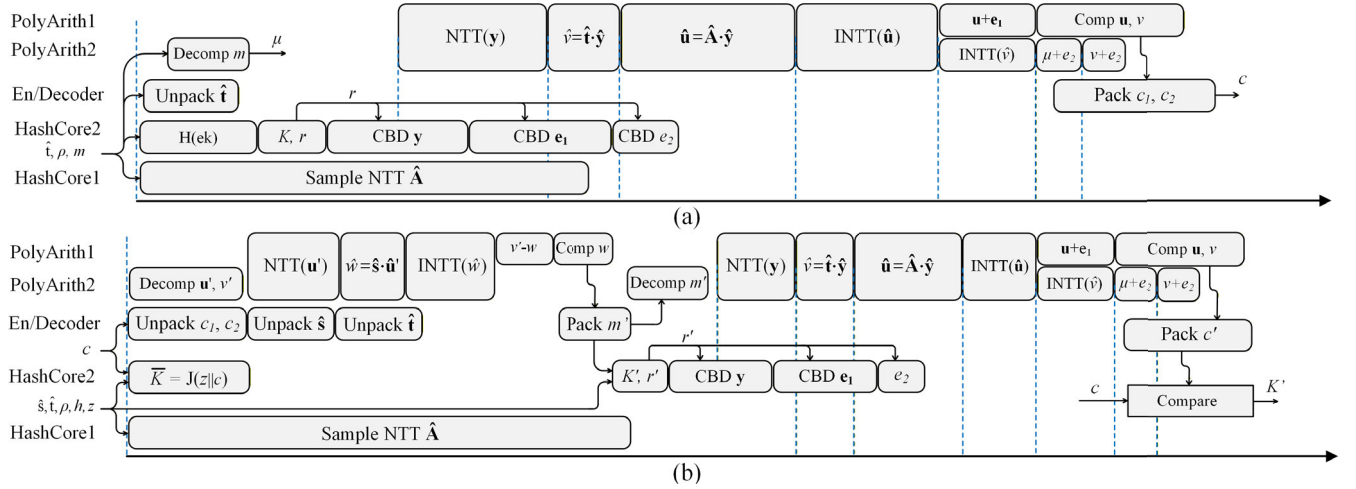


FIGURE 6. Timing schedule for ML-KEM: (a) Encapsulation (ML-KEM.Encap) and (b) Decapsulation (ML-KEM.Decap).

as the most time-consuming. Our design prioritizes the optimization of these tasks, enabling continuous execution to enhance overall performance. By focusing on these computational phases, subsequent sampling and other tasks can be effectively overlapped, thereby optimizing execution time. This design strategy reduces execution time by overlapping tasks wherever feasible, as illustrated in the operation scheduling diagrams.

2) DETAILS OF OPERATION SCHEDULING

Based on the procedural order in both algorithms and proposed unified architecture, we present an optimized timing schedule that maximizes the utilization of submodules while maintaining continuous progress along the critical operation path. During the execution of each algorithm, the controller functions as a FSM, ensuring sequential processing of all hardware modules in accordance with the algorithm's procedure order. The timing schedules corresponding to each algorithm in both standards are illustrated in Figs. 3–6. Depending on the algorithm being executed, certain processing units are configured to operate in parallel to minimize overall latency, provided no data flow conflicts arise. By optimizing task sequencing and ensuring conflict-free interconnections between submodules,

this design enables continuous execution in a pipelined manner, effectively reducing idle time and significantly enhancing overall performance.

a: ML-DSA KEY GENERATION

The FSM schedule for the key generation algorithm is delineated stage by stage in Fig. 4a. Computational, hashing, and sampling tasks occupy approximately 90% of total execution time. To boost performance, our UniPAM module, equipped with dual PolyArith units, allows simultaneous processing of two polynomials. Similarly, the hash module is configured with dual Hash Cores, with one dedicated solely to generating the public matrix A. The packing process is executed concurrently with sampling and computations. The key generation process concludes with the packing of components into public and secret keys.

b: SIGNATURE GENERATION

Employing F-SWA paradigm, this phase operates iteratively until a valid signature is generated. It comprises two phases: precomputation and a rejection sampling loop, as depicted in Fig. 3. During the precomputation phase, the secret key is unpacked, and necessary calculations are performed prior to entering the while loop specified in the

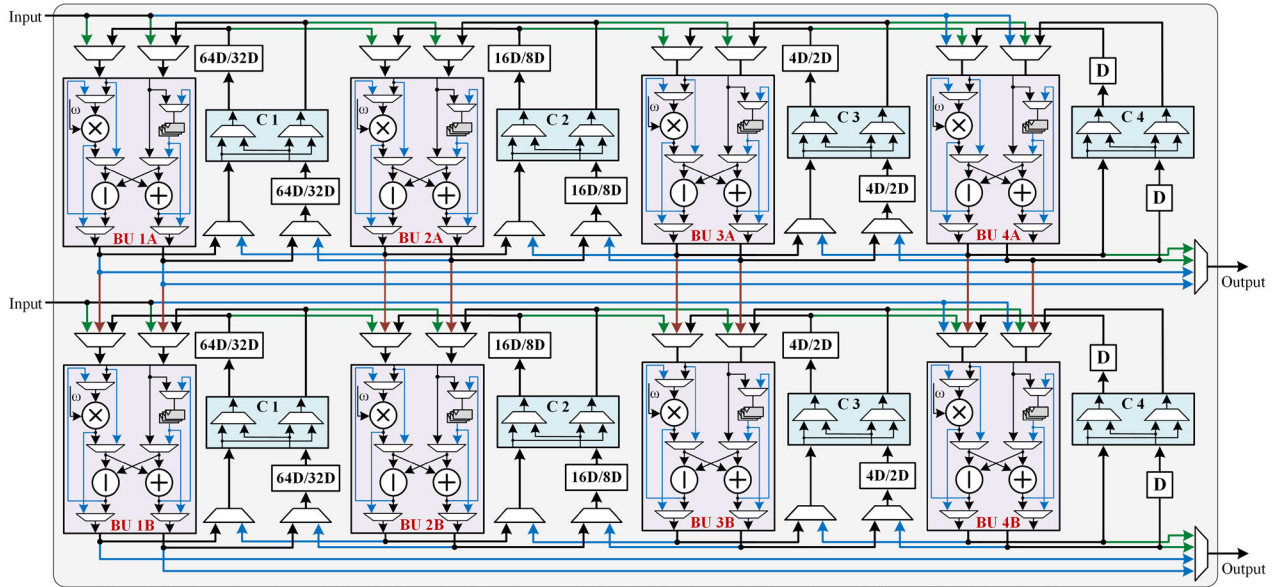


FIGURE 7. Proposed unified polynomial arithmetic module architecture.

Algorithm 1 Proposed Folding-Pipelined NTT Algorithm

Input: $a = [a[0], a[1], \dots, a[n-1]] \in \mathbb{Z}_q$

Ψ, Ψ_i contain TF constants of all stages and stage i in NTT

Output: $\hat{a} = \text{NTT}_n(a)$

```

1: In odd cycle:  $l \leftarrow \log_2(n)$ 
2: for ( $i = 1; i \leq l; i = i + 2$ ) do ▷ Unrolling
3:   for ( $j = 0; j < n; j++$ ) do ▷ Pipelining
4:      $\omega \leftarrow \Psi_i[j/2^{l-i+1}] \leftarrow \Psi[(2^{i-1} - 1) + j \cdot 2^{i-1}/n]$ 
5:      $temp = \omega \times a_{i-1}[j + n/2^i] \bmod q$ 
6:      $a_i[j + n/2^i] = (a_{i-1}[j] - temp) \bmod q$ 
7:      $a_i[j] = (a_{i-1}[j] + temp) \bmod q$ 
8:     if ( $(j + 1) \bmod (n/2^i) == 0$ ) then
9:        $j = j + n/2^i$ 
10:    end if
11:  end for
12: end for
13: In even cycle:
14: for ( $i = 2; i \leq l; i = i + 2$ ) do ▷ Unrolling
15:   for ( $j = 0; j < n; j++$ ) do ▷ Pipelining
16:      $\omega \leftarrow \Psi_i[j/2^{l-i+1}] \leftarrow \Psi[(2^{i-1} - 1) + j \cdot 2^{i-1}/n]$ 
17:      $temp = \omega \times a_{i-1}[j + n/2^i] \bmod q$ 
18:      $a_i[j + n/2^i] = (a_{i-1}[j] - temp) \bmod q$ 
19:      $a_i[j] = (a_{i-1}[j] + temp) \bmod q$ 
20:     if ( $(j + 1) \bmod (n/2^i) == 0$ ) then
21:        $j = j + n/2^i$ 
22:    end if
23:  end for
24: end for

```

ML-DSA.Sign algorithm [3]. The rejection sampling loop is responsible for generating a valid signature that satisfies the algorithm's specified requirements. Given the variable

number of iterations in the loop, execution time can become a bottleneck. To mitigate this, our design utilizes the first PolyArith unit to compute the response z and hint h , while the second unit concurrently calculates the commitment w_1 for the subsequent signature. This process continues until a valid signature is found, which is then output as a byte-encoded combination of the commitment hash, response, and hint.

c: VERIFICATION

Beginning with the unpacking of public verification key and signature, this algorithm executes a series of computations as shown in Fig. 4b. Subsequently, the signer's response z and hint h are used to reconstruct the signer's commitment w_1 , denoted as w'_1 . This reconstructed value, along with the message representative μ , serves as the seed to generate the hash $\tilde{c}'$, which is then compared with the signer's commitment hash $\tilde{c}$ to authenticate the signatory's identity.

d: ML-KEM KEYGEN

Similar to ML-DSA.KeyGen, upon receiving two random seeds as input, this algorithm proceeds through each step as described in Fig. 5 to generate and pack a key pair. With dual PolyArith units, our implementation allows concurrent execution of the NTT operations for the secret vector s and noise e at the same time. Due to the complexity of the CWM in ML-KEM, both units are combined to handle this operation, as detailed in the UniPAM architecture.

e: ENCAPSULATION AND DECAPSULATION

In encapsulation, this algorithm uses encapsulation key and message m to generate shared secret key K , which is retained by the sender. The ciphertext is then generated to encrypt and encapsulate value of m via LWE problem and sent to the receiver. Decapsulation occurs in reverse: by

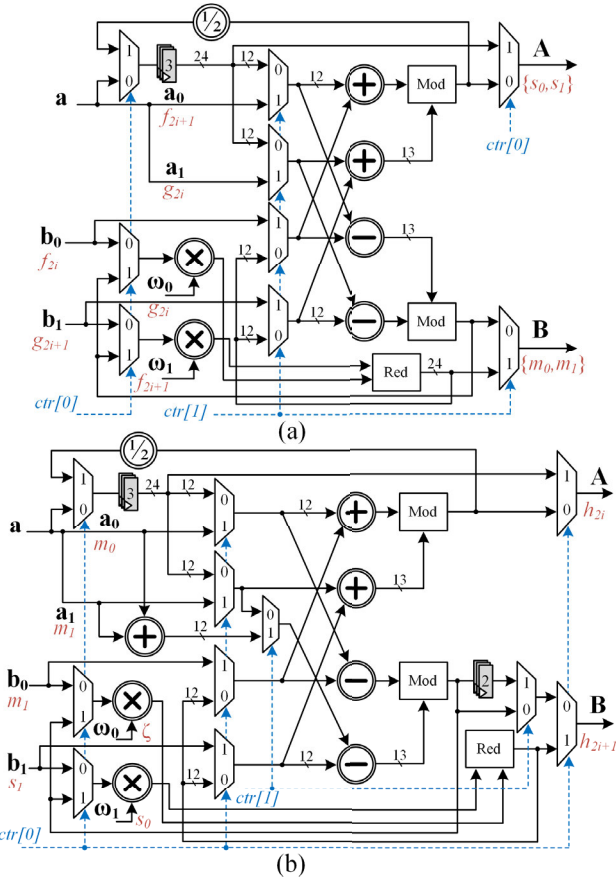


FIGURE 8. Proposed radix-2 unified BU: (a) Type-A and (b) Type-B.

parsing the components of decapsulation key and ciphertext to recover the plaintext m' , executing the encapsulation process again with m' to verify the matching of the resulting ciphertext c' with provided ciphertext c , before the receiver obtains the shared secret key K' . All procedures reenact the corresponding algorithms specified in [2], utilizing the available hardware submodules within our design. The use of dual PolyArith units and Hash Cores enhances our design's performance compared to prior studies.

B. UNIFIED POLYNOMIAL ARITHMETIC MODULE

As analyzed in the timing schedules for two algorithms, the UniPAM is a critical component that significantly influences the performance of the proposed implementation. In our preliminary analysis, we observed that in-place architectures utilizing dual radix-2 butterfly units (BUs), as employed in [28] and [29], exhibit suboptimal performance. While radix-4 and radix-2 $\times$ 2 methods, as discussed in [9] and [42], offer improvements, they are not well-suited for handling the NTT operations required by ML-KEM. Fully-pipelined NTT architectures, such as MDC and MDF architectures in [37] and [38] achieve high-speed performance for NTT/INTT operations. However, UniPAM configurations with 8 BUs are not optimal for other PAOs due to excessive memory bandwidth requirements: utilizing all 8 BUs as independent

Algorithm 2 Hardware-Friendly Compression Algorithms

- 1: $\text{Compress}_d : x \mapsto \lceil (2^d/q) \cdot x \rceil \bmod 2^d$
- 2: $t = m \cdot x$ ($m \approx (2^{35}/q) = 10321339$)
- 3: $x = (t \gg (35 - d)) + t[34 - d]$
- 4: $\text{Decompress}_d : y \mapsto \lceil (q/2^d) \cdot y \rceil$
- 5: $t = q \cdot y; \quad y = (t \gg d) + t[d - 1]$

Algorithm 3 Proposed Decompose Algorithm

Input: $r \in \mathbb{Z}_q$

Output: r_1, r_0 where $r \equiv r_1(2\gamma_2) + r_0 \bmod q$

- 1: $t = m \cdot r$ ($m \approx 2^{40}/\gamma_2$)
- 2: $r_1 = (t \gg 41) + t[40]$
- 3: **If** $(\gamma_2 = (q - 1)/32)$ $r_1 = 15$ & r_1
- 4: **Else:** $r_1 = (r_1 = 44)? 0: r_1$
- 5: $r_0 = r - r_1 \cdot (2\gamma_2) \bmod q$

PE necessitates 16 memory read ports and 8 memory write ports. Considering the structural similarities and polynomial lengths inherent in lattice-based cryptosystems, it is feasible to design a unified arithmetic architecture capable of supporting both schemes within a single implementation. Given that a 24-bit datapath suffices to store one ML-DSA coefficient or two ML-KEM coefficients, and considering the 36Kb BRAM configuration available in FPGAs [46], we configure groups of three BRAMs with a combined bank width of 96 bits to store four ML-DSA or eight ML-KEM coefficients per memory address. This efficient configuration reduces memory requirements by 25%. To leverage this approach, the number of BUs is reduced to four. Consequently, we adopt the folding transformation technique, originally introduced in folding FFT architectures [47], into our UniPAM, enabling support for NTT/INTT algorithms in both cryptosystems while halving the number of required BUs, as described in Alg. 1.

The corresponding UniPAM architecture, illustrated in Fig. 7, comprises dual PolyAriths, each leveraging 4 BUs arranged sequentially in a single chain, instead of 8 BUs used in conventional MDC architectures for 256-point NTT. Both units are designed to operate independently when handling NTT and INTT operations for both schemes. Due to the specific requirements of the CWM in ML-KEM, the two units are combined to handle this operation. To handle NTT and INTT with half the number of BUs, each BU operates in a time-sliced manner, handling two adjacent NTT stages. For instance, BU 1 processes the first and second stages during odd and even cycles, respectively. The four commutators reorder the sequence of NTT process based on each stage, as detailed in Alg. 1. For twiddle factor (TF) storage, each PolyArith employs dual-port 36Kb BRAM and eight 24-bit registers. The BRAM operates in double-output mode, delivering 72-bit TF values per CC to the last three BUs. It stores 512 precomputed TFs arranged in their order of use, supporting both schemes. This approach significantly reduces FF utilization, enhancing hardware efficiency.

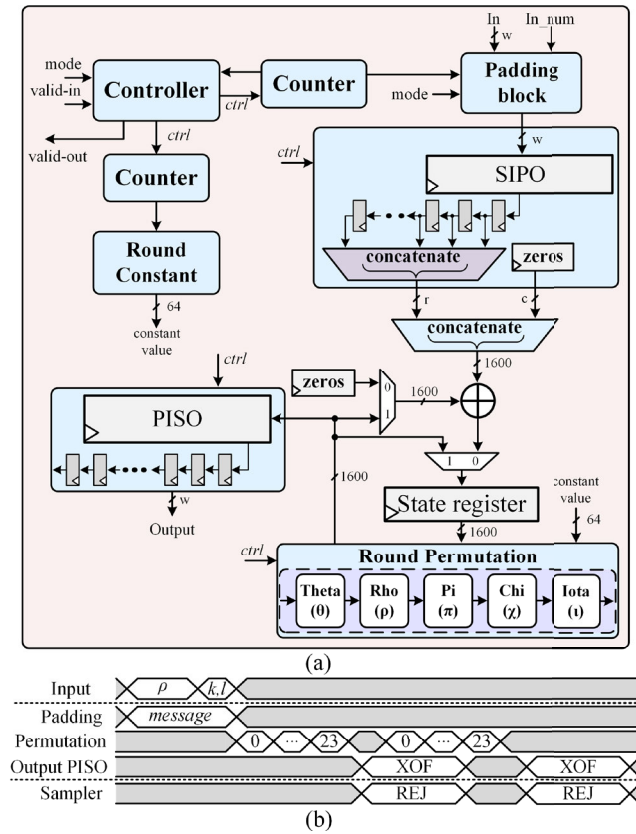


FIGURE 9. (a) Efficient hash architecture and (b) timing diagram.

In Fig. 7, green lines represent the NTT data flow, blue lines denote INTT data flow, and black lines indicate shared pathways used in both processes. The red lines denote the connection between the two PolyArith units when combined to handle the CWM of ML-KEM. When handling other PAOs, each PolyArith functions as four independent PEs to ensure a sampling rate of 4 ML-DSA and 8 ML-KEM coefficients. The folding strategy enables us to maintain a pipelined architecture while halving hardware resource consumption. As a result, our UniPAM with 4 BUs achieves nearly double the throughput of traditional architectures and avoids the complexities of address generation and data reordering commonly associated with in-place architectures like those in [28] and [29]. Additionally, the unified 4-BU design effectively balances the memory bandwidth of the NTT/INTT architecture with the computational capacity of the arithmetic units, mitigating memory access bottlenecks during concurrent polynomial transformations. To handle NTT/INTT in ML-DSA, 296 CCs are required for initial latency, which occurs only once, followed by 256 CCs for pipelined processing time per polynomial. For ML-KEM, the corresponding values are 165 CCs and 128 CCs, respectively. This unified architecture not only streamlines processing but also optimizes hardware resource utilization, ensuring efficient and secure operations across both cryptographic protocols.

1) RADIX-2 UNIFIED BUTTERFLY UNIT

Serving as the primary PE within the PolyArith, BU is crucial for executing various operations. To accommodate all PAOs in both cryptosystems, our BUs are configured to operate in multiple modes, switched via the control signal *ctr*, as depicted in the architecture in Fig. 8. In this design, the proposed BU architecture is divided into two types, corresponding to the two PolyArith units in Fig. 8. Each BU comprises two 12-bit multipliers, adders, subtractors, modular reduction (Mod) and multiplexers. These units can function independently as two ML-KEM BUs or be combined as one ML-DSA BU. The multiplexers, controlled by *ctr*, facilitate switching between operation modes, including the widely used Cooley-Tukey (CT) and Gentleman-Sande (GS) BU configurations for NTT and INTT, with corresponding *ctr* values of '00' and '11'. The CT configuration can also be utilized for multiply-accumulate (MAC) and decompose algorithm of ML-DSA, as well as addition and subtraction in both standards. To further optimize INTT, we adopt and modify the technique proposed in [43], which eliminates the need for multiplying the resulting coefficients by $n^{-1} \bmod q$ after INTT. Subsequently, divide-by-2 operation is integrated into the addition unit, and the TFs for the INTT are preprocessed to incorporate factor 2^{-1} .

As mentioned in the preliminary analysis, the CWM of ML-KEM is unique, necessitating hardware architecture designed to implement equation (1). Our UniPAM is designed to achieve sampling rate of eight coefficients for CWM in ML-KEM, with each pair of BUs handling 2 coefficients. Following this, two pairs of coefficients, f_i and g_i , are processed by type-A BU, as indicated by brown letters in Fig. 8. The intermediate results then proceed to the type-B BU in the second PolyArith, culminating in the final CWM result, h_i , at the output. Combined with simple adders at the end of the type-B BU, CWM effectively becomes MAC for ML-KEM, which can be executed in 32 CCs of pipeline time per polynomial multiplication in our UniPAM. Furthermore, by exploiting the rounding characteristics inherent in the compressions of ML-KEM and decompose algorithms in ML-DSA, these functions are integrated into the reduction unit (Red) following the multiplier, leverage existing multiplier and DSP usage, as described in Algs. 2 and 3. Integrating these functionalities into UniPAM eliminates the need for separate components, thereby reducing unnecessary hardware utilization while maintaining consistent performance.

2) MODULAR MULTIPLICATION (ModMult)

Within the BU, ModMult is complex component and significantly impacts the circuit's operating frequency. While Barrett reduction is common approach, as demonstrated in [38] and [42] for Kyber and Dilithium, it struggles with large bit-size adders required and SCAs vulnerabilities [34]. Similarly, reduction method proposed in [40], which recursively leverages modulus relation $2^{12} \equiv 2^9 + 2^8 + 1$, necessitates multiple adders for nine additions, introducing

additional complexity. To address these challenges, we developed an alternative approach for ML-KEM. Observing that the 24-bit result from multiplication can be expressed as the sum of four 12-bit terms, ModMult can be efficiently represented as:

$$a[23 : 0] \pmod{q} \equiv 2^{20}.a[23 : 20] + 2^{16}.a[19 : 16] \\ + 2^{12}.a[15 : 12] + a[11 : 0] \pmod{q}$$

The results of 4-bit modulus operation can be precomputed and stored in two 16-to-1 multiplexers (16-Muxs) and one 10-Mux. After one more reduction after addition, the final result of ModMult can be derived straightforwardly. For ML-DSA, we adopt and modify the modulus relation method proposed in [14], which recursively exploits the modular relation. This approach significantly reduces the number and size of adders required compared to Barrett reduction in [42], avoiding large bit-size adders. To accommodate the 25×18 -bit multipliers available in DSP blocks, the unified 24×24 -bit multiplier is divided into 2 segments. The computation is expressed as:

$$a[23 : 0] \times b[23 : 0] = 2^{12}.a[23 : 0] \times b[23 : 12] \\ + a[23 : 0] \times b[11 : 0] = m_0 + 2^{12}.m_1$$

The two multiplications are executed on separate DSPs, resulting in high-bit and low-bit multiplication outputs, m_0 and m_1 . By recursively reducing m_0 and m_1 using the relation ($2^{23} \equiv 2^{13} - 1$), the final result of 24-bit multiplication result is confined within the range $(-q, 2q)$. A subsequent reduction step then produces the ModMult result for ML-DSA:

$$2^{12}.m_1 + m_0 \equiv m_0[22 : 0] + 2^{13}(m_1[33 : 31] + m_1[30 : 21] \\ + m_1[20 : 11] + m_0[34 : 33] + m_0[32 : 23]) \\ + 2^{12}.m_1[10 : 0] - (m_1[33 : 31] + m_1[33 : 21] \\ + m_1[33 : 11] + m_0[34 : 33] + m_0[34 : 23])$$

C. HASHING AND SAMPLING MODULE

The two cryptographic systems utilize a total of four instances of the SHA-3 standard [4], necessitating a hash module capable of supporting these functions. All instances share the same sponge construction, wherein data is absorbed into the sponge and subsequently squeezed out. Despite differences in internal parameters—bit-rate r and capacity c —they all employ the same permutation function, known as *Keccak-p*. This function comprises 24 round permutations, each consisting of five step mappings: θ , ρ , π , χ and ι , as illustrated in Fig. 9. The commonality of *Keccak-p* function across all instances allows for the design of a flexible hash architecture capable of handling multiple hash instances. The architecture of our Hash module is depicted in figure 2 and comprises control unit and one or more Keccak datapath units, each incorporating *Keccak-p* function. Given the large number of polynomials in the public matrix $\hat{A}$, sampling this matrix is time-consuming. To address this, and to meet

high-performance targets, our design employs dual Keccak datapath units in the HashCore 1 exclusively for generating this matrix, thereby accelerating the execution time of these cryptosystems. For pseudorandom vector sampling and hashing tasks used in both standards, we configure HashCore 2 with a single permutation function. This configuration allows for flexible adjustment of the parameters r and c to accommodate other SHA3 instances.

The overall architecture of Hash module and its role within our system are described in Figs. 2 and 9. In this setup, the controller receives signals indicating mode of operation and the number of bytes in absorbing and squeezing phases, which are necessary to control the Keccak datapath operating as *Keccak-p* during both phases. The Keccak datapath comprises padding block, serial-in-parallel-out (SIPO), parallel-in-serial-out (PISO), and round permutation units. Depending on the instance mode, it reconfigures parameters to ensure proper operation of *Keccak-p*. Padding block receives the seed as input and stores it in the state register for *Keccak-p*. In SHAKE128, bit-rate r is 1344; therefore, HashCore 1, used for this function, has chosen bandwidth w of 96 bits. Conversely, HashCore 2 serves SHAKE256, SHA3-256, and SHA3-512, which have r values of 1088 or 576. Consequently, bandwidth w for HashCore 2 is set to 64 bits, which is the greatest common divisor and is also suitable for interfacing with other submodules in our implementation. The SIPO and PISO units are used as the temporary registers to connect the Hash core's bandwidth with the state of *Keccak-p*.

Aiming for high-performance design, the timing schedule of our Hash module is also designed to enhance its functionality. For instance, when sampling matrix $\hat{A}$ by XOF, the task begins with the padding block taking 5 CCs to read seed ρ and 16-bit nonce. Subsequently, the *Keccak-p* function executes 24-round permutations over 24 CCs. The sponge construction then enters the squeezing phase, where the r -bits truncation of the Keccak state is output as pseudorandom values for the matrix $\hat{A}$, while the *Keccak-p* immediately continues to generate the next permutation, as described in Fig. 9. The pseudorandom values are passed to the sampler rejection module to verify their validity before being grouped into coefficients and stored in the polynomial memory. This process occurs concurrently with the next *Keccak-p* execution to optimize execution time. The acceptance rates of coefficients in the matrix $\hat{A}$ for ML-DSA and ML-KEM are 99.9% and 81.27%, requiring an average of 5 and 3 permutations per $\hat{A}$ polynomial. Consequently, an average of 147 and 100 CCs are needed to generate two $\hat{A}$ polynomials in the two standards, due to our HashCore 1 having dual Keccak datapath units. A similar method is employed in HashCore 2 to generate pseudorandom vectors. For certain hashing tasks involving large seed sizes, such as the functions $H(ek)$ and $H(pk)$ in the Keygen processes of both standards, the absorbing phase is also executed concurrently with *Keccak-p* to optimize execution time.

TABLE 2. Resource utilization of proposed architecture on the ZYNQ Ultrascale+ ZCU102 FPGA platform.

Submodule	Resource Utilization			
	LUT	FF	DSP	RAM
Polynomial RAM	5,099	0	0	25.5
Hint	9,133	3,747	0	0
En/Decoder	4,744	701	0	0
Composer	1,565	728	0	0
UniPAM	10,904	4,442	16	2
HashCore 1	9,157	6,311	0	0
HashCore 2	9,426	3,909	0	0
Sampling	13,366	3,081	0	0
FSM & Others	3,916	824	0	0
Combined Architecture	67,310	23,743	16	27.5

D. SIDE-CHANNEL ATTACKS AND COUNTERMEASURES

Even though PQC algorithms are proven secure against attacks from quantum computers, SCAs remain a significant threat to any cryptographic scheme, including PQC. SCAs can exploit leakages to recover secret keys, plaintexts, or induce faults in deterministic signatures. To counter these risks, in addition to the theoretical improvements integrated into the latest versions of ML-KEM and ML-DSA, practical countermeasures must be applied in hardware implementations. Cryptanalysis studies [31], [32], [33], [34] reveal that PQC hardware implementations still exhibit vulnerabilities against electromagnetic analysis, simple power analysis, and correlation power analysis. However, these vulnerabilities are often overlooked in previous hardware implementations of Kyber and Dilithium. To address this issue, our hardware architecture adopts a hiding countermeasure approach to partially mitigate SCAs, as described in [31], [32], [33], and [34], thereby enhancing the overall security of the system. According to the secure usage guidelines for KEMs and DSAs [49], [50], device and channel security must be ensured, while advanced countermeasures such as masking or shuffling—though effective—can significantly impact resource utilization and circuit timing. In our design, we rely primarily on the hiding countermeasure method, combined with experiences learned from the vulnerable phases of ML-DSA and ML-KEM, to propose several practical solutions against SCAs while maintaining efficiency and performance comparable to previous implementations that did not incorporate SCA protections.

Recent studies [31], [32], [33] indicate that the most vulnerable operations are those involving the sampling and manipulation of the secret vector $\mathbf{s}$. To mitigate this, our architecture introduces parallelism and noise injection via concurrent task execution. As illustrated in Fig. 3, during decoding and NTT of secret vectors $\mathbf{s}_1$, $\mathbf{s}_2$, we execute parallel operations such as double sampling of the matrix $\hat{\mathbf{A}}$ in HashCore1, as well as hashing and sampling of vector $\mathbf{y}$ in HashCore2, alongside NTT computations of $\mathbf{y}$ in arithmetic unit 2. Similarly, in Fig. 4a, sensitive tasks like packing in the encoder and the generation of vector $\mathbf{s}_1$ are interleaved with other operations, introducing noise to resist EM and power

TABLE 3. Performance results of ML-DSA and ML-KEM.

Ref	Parameter set	Keygen		Sign*/Encap		Verify/Decap	
		kCCs	μs	kCCs	μs	kCCs	μs
TW	ML-DSA-44	3.18	10.6	10.48/28.99	34.93/96.63	3.29	10.97
	ML-DSA-65	5.02	16.73	14.41/45.94	48.03/153.13	5.57	18.55
	ML-DSA-87	8.24	27.45	20.61/49.99	68.7/166.64	8.0	26.66
	ML-KEM-512	0.89	2.97	1.47	4.9	2.18	7.27
	ML-KEM-768	1.19	3.97	2.0	6.67	2.84	9.46
	ML-KEM-1024	1.55	5.17	2.44	8.13	3.31	11.03
[29]	Dilithium-2	14.59	54.05	31.66/-	117.27/-	15.42	57.12
	Dilithium-3	23.62	87.48	48.45/-	179.43/-	26.12	96.76
	Dilithium-5	39.74	147.18	70.18/-	259.92/-	46.67	172.86
	Kyber-512	3.4	12.57	4.96	18.36	6.81	25.21
	Kyber-768	6.29	23.3	7.86	29.12	11.29	41.82
	Kyber-1024	9.09	33.66	11.35	42.04	13.9	51.5

* Results for best-case/average-case scenarios.

analysis traces. In ML-KEM KeyGen (Fig. 5), generation of secret vector $\mathbf{s}$ and noise $\mathbf{e}$ is interleaved—e.g., generating polynomial $\mathbf{s}_0$, then $\mathbf{e}_0$, continuing with $\mathbf{s}_i$ and $\mathbf{e}_i$. Their NTT computations are also processed concurrently. This parallel execution conceals leakage from sensitive values, strengthening resistance against SCAs. A similar strategy is applied during decapsulation, where decoding and $\mathbf{s}$ -related computations are overlapped with $\hat{\mathbf{A}}$ generation, arithmetic, and hashing, effectively masking leakage and raising security.

Derived from the demonstration in [33], the NTT process can still be attacked even under noisy conditions, particularly in the first stage where sensitive information from secret vectors is exposed through multiple signings under the same key pair. To mitigate this, instead of using in-place NTT architectures proposed in previous works—which execute computations fully iteratively—our UniPAM adopts a folding-pipelined architecture. In this design, a half stages are executed in parallel within a group, and these groups are then processed sequentially. This hybrid approach significantly reduces the exposure of first-stage inputs to EM near-field probes in close proximity to the device [31], [33]. Moreover, we propose a modular multiplication method different from the traditional Barrett reduction, which has been shown vulnerable to chosen-ciphertext clustering attacks exploiting side-channel leakage during the decapsulation phase [34].

For ML-KEM, our timing diagrams (Figs. 5 and 6) are carefully designed to ensure near-constant time execution in almost all cases, thereby mitigating timing attacks described in [48]. In ML-DSA, although execution time may vary depending on message length due to rejection sampling, all other segments are executed with consistent timing, including each iteration of the signing loop. Additionally, consistent with secure usage guidelines for KEM and DSA [49], [50], shared secrets, decapsulation keys, and input messages are immediately packaged and transmitted after generation to avoid memory retention. Similarly, memory regions holding

TABLE 4. Comparison of implementation results between the proposed design and previous works at the highest security level.

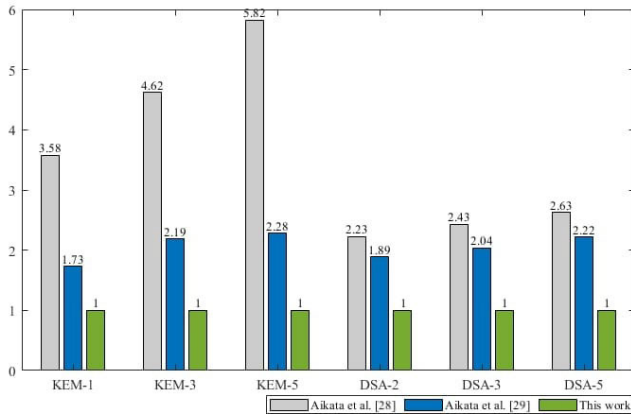
Ref.	Scheme	Platform	Utilization				Freq. [MHz]	Operations Time ¹ [kCCs]	T. Time ² [μs]	EqS ³	ATP ⁴ (Ratio)
			LUT	FF	DSP	BRAM					
[9]	Dilithium	Virtex US+	53,907	28,435	16	29	256	14.04/24.36/50.07/14.64	207.11	12,705	2.63
[13]	Dilithium	Zynq US+	32,035	9,707	24	14	300	10.86/20.33/55.17/12.66	146.17	7,467	1.09
[14]	ML-DSA	Virtex US+	49,753	23,477	16	27.5	300	8.22/21.2/50.85/8.06	124.93	11,701	1.46
[19]	Kyber	Artix-7	8,112	6,077	4	5.5	311	6.3/8.2/9.9	78.5	4,475	0.35
[23]	ML-KEM	Artix-7	25,757	18,629	21	14	210	3.40/3.50/5.80	60.47	14,172	0.86
[24]	ML-KEM	Artix-7	7,600	5,700	4	3	169	4.5/5.1/7.6	101.8	3,719	0.38
[25]	ML-KEM	Artix-7	12,037	6,895	4	9	220	4.6/6.1/8.0	85	6,372	0.54
[28]	Saber& Dilithium	Zynq US+	18,494	9,319	4	24	200	17.52/21.60/27.89 38.84/68.46/ - /45.79	335.05 765.45	5,888	6.48 (×3.16)
[29]	Kyber& Dilithium	Zynq US+	23,277	9,798	4	24	270	6.81/11.29/13.91 39.74/70.18/ - /46.67	118.56 579.96	6,516	4.55 (×2.22)
TW	ML-KEM& ML-DSA	Zynq US+	67,310	23,743	16	27.5	300	1.55/2.44/3.31 8.24/20.61/49.99/8.0	24.33 122.83	13,912	2.05 (1.0)

¹ In order of Keygen/ Best-case and average-case scenarios of Sign/ Verify in ML-DSA and Keygen/ Encap/ Decap in ML-KEM.

² Total execution time of three main operations in KEM and the best-case scenario in DSA.

³ Equivalent Slices (EqS) = #Slice + Eq. DSP + Eq. BRAM. In which, #Slice $\approx \frac{\#LUT}{4} + \frac{\#FF}{8}$ (7 series)/ $\frac{\#LUT}{8} + \frac{\#FF}{16}$ (UltraScale)
 DSP ≈ 102.4 Slices (7 series)/51.2 Slices (UltraScale), 36Kb BRAM ≈ 232.4 Slices (7 series)/ 116.2 Slices (UltraScale) [53], [54].

⁴ Area-time product (EqS×s) = EqS × Total Time and ratio between this work (TW) and others.

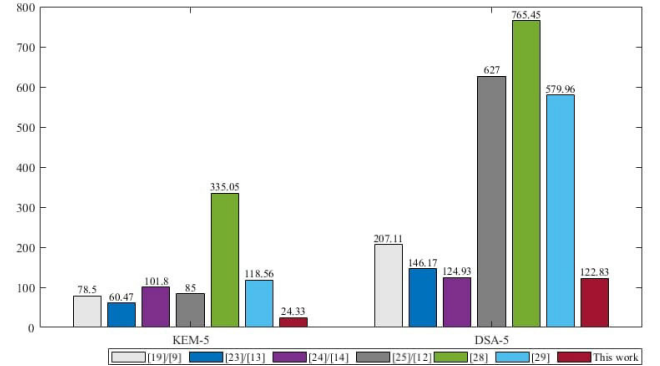
**FIGURE 10.** Comparison of Area-Time product metric (EqS × s).

secret vectors are promptly overwritten with non-sensitive data once computations are complete, preventing leakage through memory analysis.

IV. IMPLEMENTATION RESULTS AND COMPARISONS

A. RESOURCES UTILIZATION AND PERFORMANCE RESULTS

To evaluate the efficiency and performance of our unified architecture, we implemented the design using Verilog-HDL on the Xilinx Vivado 2023.2 design suite. The implementation was synthesized and placed-and-routed on a Xilinx Zynq UltraScale+ ZCU102 FPGA platform (xczu9eg-ffvb1156-2-e). Test vectors were sourced from the NIST Cryptographic Algorithm Validation Program, which provides standardized test vectors for validating cryptographic algorithm implementations [35]. Given the variations in hardware platforms and

**FIGURE 11.** Comparison of performance results (μs).

resource utilization across different studies, we employed the Area-Time Product (ATP) and Equivalent number of Slices (EqS) as primary metrics for efficiency evaluation. These metrics, widely adopted in prior research [51], [52], account for equivalencies between FPGA components such as BRAMs, DSPs, and slices, providing a standardized basis for comparison. Aiming for high-performance architecture, we meticulously optimized operations using hardware-friendly techniques. This included inserting pipeline registers to break long combinational paths, reducing critical path delays, and balancing pipeline stages to ensure uniform delays and prevent bottlenecks. As a result, our design achieved a maximum operating frequency of 300 MHz and total power consumption is 6.86W on the Zynq US+ platform. The detailed resource utilization of the complete design and its main submodules is summarized in Table 2.

In terms of performance, due to the rejection conditions in sampling pseudorandom matrices and vectors, as well as the

variable message sizes in ML-DSA, execution times for each algorithm exhibit slight variations. Based on data collected from multiple samples, the average performance statistics for each algorithm across three security categories are presented in Table 3, alongside comparisons with the most related study. Notably, the signature generation process employs the F-SWA paradigm, requiring variable iterations of the rejection loop until a valid signature is found. To measure the execution time of this algorithm, as outlined in its timing schedule (Fig. 3), we divided it into precomputation and rejection loop phases. In our design, the average time per attempt in the rejection loop phase is 5696, 7692, and 10312 CCs for three security categories, respectively. The performance of this phase for best-case and average-case scenarios are reported in Table 3, with the message size and context string fixed at 32 bytes. For longer messages, an additional 24 CCs are required for each 136 bytes beyond the initial 32 bytes in categories level 2 and 3. At the highest security level, this increment applies after the first 1632 bytes of the message. Based on the expected number of repetitions of the main loop in the signing algorithm—4.25, 5.1, and 3.85—the average execution times for the signature generation algorithm are 28990, 45944, and 49994 CCs for the respective security categories. Similarly, for the Verify algorithm, the data presented in Table 3 covers cases where the combined size of the message and context string is less than 18496, 22304, and 33320 bytes for the three security categories, respectively. Our implementation requires an additional 24 CCs for every 136 bytes processed, corresponding to 1 absorbing round of SHAKE256 function.

B. COMPARISON AND DISCUSSION

To assess the efficiency of our proposed implementation against state-of-the-art (SOTA) studies, we compiled and compared implementation results in Table 4. For a fair comparison, we selected several of the most relevant and high-performing FPGA implementations supporting the three security categories of ML-KEM and ML-DSA, as well as combined implementations. Previous combined architectures did not address the repetitions of the main loop in the signing algorithm and only reported results for best-case scenarios (i.e., when a valid signature is generated on the first attempt). Therefore, we also considered best-case scenarios to evaluate the ATP parameter across all implementations. Prioritizing high-performance targets, we present comparisons of ATP and performance results for the highest security category in figures 10 and 11 for clarification. Based on these implementation results, our implementation demonstrates superior performance compared to the work in [9] across most comparison parameters, particularly in ATP and overall performance. When compared with the implementation by Li et al. [13], our design exhibits higher area utilization but achieves better performance, especially in the KeyGen and Verify algorithms.

Regarding Kyber and ML-KEM implementations, the hardware architecture for these standards requires

significantly less time and area compared to the ML-DSA standard. Therefore, we focused on evaluating performance results, which are crucial for fast implementations in practical internet applications and a primary motivation for hardware implementation of these standards. In this context, our implementation significantly improves execution times for all main algorithms across the three security categories. Notably, at the highest security level, our implementation is $2.49\times$ faster than the ML-KEM design in [23] and over $3\times$ faster than the works in [24] and [25]. The superior performance of our implementation compared to most SOTA studies is illustrated in figure 11 and Table 4, with an average of 3 times faster execution across all main algorithms.

The development of combined architectures for DSA and KEM is still evolving. To our knowledge, only the combined architectures by Aikata et al. have been proposed for Dilithium with Saber in [28] and Dilithium with Kyber in [29]. These lightweight architectures focus on minimal hardware consumption, utilizing a single hash module and an arithmetic module with double butterfly units. By employing double arithmetic units with four BUs and hash module featuring dual HashCores, our design significantly boosts execution time. Consequently, in terms of performance, our implementation shows significant improvements, achieving over $4\times$ faster execution for ML-KEM and ML-DSA across all security levels. Additionally, our hardware architecture provides a detailed solution to handle the rejection loop in the signature generation algorithm, which was not addressed in previous studies. Our paper is also the first to propose and analyze an appropriate timing schedule for each algorithm of both cryptosystems. By carefully arranging the timing schedule based on the time requirements of each algorithmic step and the sampling rates of submodules, our implementation significantly enhances efficiency, achieving an ATP parameter outperforming the most related implementation [29] by 2.22 times. These improvements position our design as a leading solution for high-performance post-quantum cryptographic applications. In addition to the RTL-based FPGA architecture, we are also developing an ASIC implementation to demonstrate the practicality of our design for real-world PQC processors. Our work is currently being extended toward the development of a commercial PQC chip in the near future.

V. CONCLUSION

In this paper, we present a high-performance unified hardware architecture that integrates both ML-DSA and ML-KEM standards, supporting all primary algorithms across the three NIST-defined security levels. This work represents the first implementation aligned with the official NIST standards, distinguishing it from prior designs based on Kyber and Dilithium submissions. The design is ready for deployment on FPGA platforms, offering flexibility and versatility through configurable operation modes and security levels. Data handling is optimized with 8-byte width packing/unpacking, facilitating seamless integration

with other software and hardware platforms. Furthermore, by employing a timing schedule that combines multiple versatile submodules, our implementation demonstrates superior performance compared to SOTA studies, establishing it as the highest-performing implementation for both cryptosystems. Additionally, efficiency evaluations using the ATP metric highlight the advantages of our implementation over previous unified hardware designs for DSA and KEM. In summary, our proposed architecture exhibits adaptability to diverse applications of KEM and DSA, providing resilience against both quantum and classical attacks.

REFERENCES

- [1] *Status Report on the Third Round of the NIST-PQC Standardization Process*. Accessed: Jul. 2022. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [2] NIST, Gaithersburg, MD, USA. *FIPS 203—Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Accessed: Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [3] *FIPS204-Module-Lattice-Based Digital Signature Standard*. Accessed: Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [4] *FIPS 202—SHA3 Standard: Permutation-Based Hash and Extendable-Out Function*. Accessed: Aug. 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [5] S. Ricci, L. Malina, P. Jedlicka, D. Smekal, J. Hajny, P. Cibik, and P. Dobias, "Implementing CRYSTALS-dilithium signature scheme on FPGAs," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, no. 1, Aug. 2021, pp. 1–11.
- [6] G. Land, P. Sasdrich, and T. Güneysu, "Hard crystal-implementing dilithium on reconfigurable hardware," in *Proc. 20th Int. Conf. Smart Card Res. Adv. Appl. (CARDIS)*, Nov. 2021, pp. 210–230.
- [7] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for CRYSTALS-dilithium," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 270–295, Nov. 2021.
- [8] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for CRYSTALS-dilithium," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2024, pp. 150–159.
- [9] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of CRYSTALS-dilithium," in *Proc. Int. Conf. Field-Program. Technol. (ICFPT)*, Dec. 2021, pp. 1–10.
- [10] L. Beckwith, D. T. Nguyen, and K. Gaj, "Hardware accelerators for digital signature algorithms dilithium and FALCON," *IEEE Design Test*, vol. 41, no. 5, pp. 27–35, Oct. 2024.
- [11] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of dilithium signature scheme on FPGA SoC platform," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 9, pp. 1158–1171, Sep. 2022.
- [12] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 8, pp. 3234–3243, Aug. 2023.
- [13] X. Li, J. Lu, D. Liu, A. Li, S. Yang, and T. Huang, "A high speed post-quantum crypto-processor for crystals-dilithium," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2024.
- [14] Q. D. Truong, P. N. Duong, and H. Lee, "Efficient low-latency hardware architecture for module-lattice-based digital signature standard," *IEEE Access*, vol. 12, pp. 32395–32407, 2024.
- [15] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, Feb. 2021, pp. 328–356.
- [16] V. B. Dang, K. Mohajerani, and K. Gaj, "High-speed hardware architectures and FPGA benchmarking of CRYSTALS-kyber, NTRU, and saber," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 306–320, Feb. 2023.
- [17] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "Instruction-set accelerated implementation of CRYSTALS-kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.
- [18] Z. Ni, A. Khalid, D.-E.-S. Kundi, M. O'Neill, and W. Liu, "HPKA: A high-performance CRYSTALS-kyber accelerator exploring efficient pipelining," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3340–3353, Dec. 2023.
- [19] Y. Cui, J. Chen, Z. Ni, Z. Zhang, C. Wang, and W. Liu, "Instruction-based high-performance hardware controller of CRYSTALS-kyber with balanced resource utilization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2394–2407, May 2025.
- [20] W. Guo and S. Li, "Highly-efficient hardware architecture for CRYSTALS-kyber with a novel conflict-free memory access pattern," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 11, pp. 4505–4515, Nov. 2023.
- [21] W. Guo and S. Li, "Split-radix based compact hardware architecture for CRYSTALS-kyber," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 97–108, Jan. 2024.
- [22] T.-H. Nguyen, D.-T. Dam, P.-P. Duong, B. Kieu-Do-Nguyen, C. Pham, and T.-T. Hoang, "Efficient hardware implementation of the lightweight CRYSTALS-kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 1–13, Feb. 2024.
- [23] H. Kim, H. Jung, A. Satriawan, and H. Lee, "A configurable ML-KEM/Kyber key-encapsulation hardware accelerator architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 11, pp. 4678–4682, Nov. 2024.
- [24] T.-H. Nguyen, T.-K. Dang, D.-T. Dam, K.-D. Nguyen, P.-P. Duong, C.-K. Pham, and T.-T. Hoang, "An area-time efficient hardware architecture for ML-KEM post-quantum cryptography standard," *IEEE Access*, vol. 13, pp. 103834–103847, 2025.
- [25] H. Jung, Q. Dang Truong, and H. Lee, "Highly-efficient hardware architecture for ML-KEM PQC standard," *IEEE Open J. Circuits Syst.*, vol. 6, pp. 356–369, 2025.
- [26] S. Mandal and D. B. Roy, "KiD: A hardware design framework targeting unified NTT multiplication for CRYSTALS-kyber and CRYSTALS-dilithium on FPGA," in *Proc. 37th Int. Conf. VLSI Design 23rd Int. Conf. Embedded Syst. (VLSID)*, India, Jan. 2024, pp. 455–460.
- [27] Q. D. Truong and H. Lee, "Efficient polynomial arithmetic and hash modules for ML-DSA and ML-KEM standards," in *Proc. IEEE Asia-Pacific Conf. Circuits Syst. (APCCAS)*, Taiwan, Nov. 2024, pp. 776–780.
- [28] A. Aikata, A. C. Mert, D. Jacquemin, A. Das, D. Matthews, S. Ghosh, and S. S. Roy, "A unified cryptoprocessor for lattice-based signature and key-exchange," *IEEE Trans. Comput.*, vol. 72, no. 6, pp. 1568–1580, Jun. 2023.
- [29] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, "KaLi: A crystal for post-quantum security using kyber and dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 747–758, Feb. 2023.
- [30] V. Lyubashevsky, "Fiat-shamir with aborts: Applications to lattice and factoring-based signatures," in *Proc. Adv. Cryptol. (ASIACRYPT)*, 2009, pp. 598–616.
- [31] Y. Zhao, S. Pan, H. Ma, Y. Gao, X. Song, J. He, and Y. Jin, "Side channel security oriented evaluation and protection on hardware implementations of kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5025–5035, Dec. 2023.
- [32] E. Karabulut, E. Alkim, and A. Aysu, "Single-trace side-channel attacks on ω -small polynomial sampling: With applications to NTRU, NTRU prime, and CRYSTALS-DILITHIUM," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, Dec. 2021, pp. 35–45.
- [33] H. M. Steffen, G. Land, L. J. Kogelheide, and T. Güneysu, "Breaking and protecting the crystal: Side-channel analysis of dilithium in hardware," in *Proc. Int. Conf. Post-Quantum Cryptography*, 2023, pp. 688–711.
- [34] B.-Y. Sim, A. Park, and D.-G. Han, "Chosen-ciphertext clustering attack on CRYSTALS-KYBER using the side-channel leakage of Barrett reduction," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21382–21397, Nov. 2022.
- [35] (May 2025). *Automated Cryptographic Validation Test System-Gen/Vals*. [Online]. Available: <https://github.com/usnistgov/ACVP-Server>
- [36] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Proc. Int. Conf. Parallel Process.*, Apr. 1996, pp. 766–770.
- [37] T.-H. Nguyen, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "High-speed NTT accelerator for CRYSTAL-kyber and CRYSTAL-dilithium," *IEEE Access*, vol. 12, pp. 34918–34930, 2024.
- [38] P. Duong-Ngoc and H. Lee, "Configurable mixed-radix number theoretic transform architecture for lattice-based cryptography," *IEEE Access*, vol. 10, pp. 12732–12741, 2022.

- [39] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, pp. 94–126, Nov. 2021.
- [40] F. Yaman, A. C. Mert, E. Öztürk, and E. Savas, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-KYBER PQC scheme," in *Proc. Design, Autom. Test Eur. Conf. Exhi. (DATE)*, Feb. 2021, pp. 1020–1025.
- [41] K. Javeed and D. Gregg, "Efficient number theoretic transform architecture for CRYSTALS-kyber," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 1, pp. 263–267, Jan. 2025.
- [42] T. X. Pham, P. Duong-Ngoc, and H. Lee, "An efficient unified polynomial arithmetic unit for CRYSTALS-dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4854–4864, Dec. 2023.
- [43] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, "Highly efficient architecture of NewHope-NIST on FPGA using low-complexity NTT/INTT," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 2, pp. 49–72, Mar. 2020.
- [44] S. Mandal and D. Basu Roy, "Winograd for NTT: A case study on higher-radix and low-latency implementation of NTT for post quantum cryptography on FPGA," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 12, pp. 6396–6409, Dec. 2024.
- [45] Q. Dang Truong, P. Duong-Ngoc, and H. Lee, "Hybrid number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 7, pp. 2039–2043, Jul. 2025.
- [46] *UltraScale Architecture Memory Resources: User Guide*, UG573 (v1.13), AMD Xilinx, Sep. 2021.
- [47] K. K. Parhi, C.-Y. Wang, and A. P. Brown, "Synthesis of control circuits in folded pipelined DSP architectures," *IEEE J. Solid-State Circuits*, vol. 27, no. 1, pp. 29–43, Jan. 1992.
- [48] P. Kocher, "Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems," in *Proc. Annu. Int. Cryptol. Conf. Adv. Cryptol. (CRYPTO)*, Aug. 1996, pp. 104–113.
- [49] NIST, *Recommendations for Key-Encapsulation Mechanisms*, Standard (SP) 800–227, 2025. [Online]. Available: <https://csrc.nist.gov/publications>
- [50] E. Barker, *Recommendation for Obtaining Assurances for Digital Signature Applications*, Standard 800–89, 2006, doi: [10.6028/NIST.SP.800-89](https://doi.org/10.6028/NIST.SP.800-89).
- [51] D.-E.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, and W. Liu, "Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 4, pp. 1993–2005, Oct. 2022.
- [52] P. Duong-Ngoc, S. Kwon, D. Yoo, and H. Lee, "Area-efficient number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 3, pp. 1270–1283, Mar. 2023.
- [53] *7 Series FPGAs Configurable Logic Block: User Guide*, UG474 (v1.9), AMD Xilinx, Apr. 2025.
- [54] *UltraScale Architecture and Product Data Sheet: Overview*, AMD Xilinx, Product Specification DS890 (v4.7), May 2025.



QUANG DANG TRUONG (Graduate Student Member, IEEE) received the B.S. degree in electrical and electronic engineering from Le Quy Don University of Technology, Vietnam, in 2019, and the M.S. degree in electrical and computer engineering from Inha University, in 2023, where he is currently pursuing the Ph.D. degree. His research interests include algorithm and architecture design for post-quantum cryptography, homomorphic encryption, and digital signal processing.



YUNSEONG JANG (Graduate Student Member, IEEE) received the B.S. degree in electronic engineering from Inha University, Incheon, South Korea, in 2025, where he is currently pursuing the M.S. degree in electrical and computer engineering. His research interests include algorithm and architecture design for post-quantum cryptography, hardware security, and digital integrated circuits and systems.



HANHO LEE (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in electrical and computer engineering from the University of Minnesota, Minneapolis, USA, in 1996 and 2000, respectively. In 1999, he was a Member of Technical Staff-1 with Lucent Technologies, Bell Labs, Holmdel, NJ, USA. From April 2000 to August 2002, he was a Member of Technical Staff with Lucent Technologies (Bell Laboratories Innovations), Allentown, USA, where he was involved in the design of DSP multiprocessor architecture. From August 2002 to August 2004, he was an Assistant Professor with the Department of Electrical and Computer Engineering, University of Connecticut, USA. He has been a Faculty Member with Inha University, Incheon, South Korea, since September 2004, initially with the Department of Information and Communication Engineering, and, since 2025, with the Department of Electrical and Electronic Engineering, where he is currently a Full Professor. He leads the Digital Integrated Systems Laboratory. He was a Visiting Researcher with the Electronics and Telecommunications Research Institute, South Korea, in 2005. He was a Visiting Scholar with Bell Laboratories, Alcatel-Lucent, Murray Hill, USA, from 2010 to 2011, and a Visiting Professor with The University of Texas, Dallas, USA, from 2017 to 2018. He is the Director of the Artificial Intelligence System on Chip Research Center, Inha University. His research interests include algorithm and VLSI architecture design for post-quantum cryptography, homomorphic encryption, artificial intelligence, forward error correction coding, and digital signal processing. He served as the General Chair for ISICAS and the Technical Program Chair for ISCAS and APCCAS. He was the Chair of the IEEE Circuits and Systems for Communications Technical Committee. He was a Board of Governor of the IEEE Circuits and Systems Society (CASS), from 2020 to 2023. He is the Vice President of Technical Activities of the IEEE CASS.

...